МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ЗВІТ

Комп’ютерний практикум №3

«Тестування програмного забезпечення»

з дисципліни

«Технології розробки WEB-орієнтованих систем»

Тема №69: «Спортзал»
Web-застосунок «Ритм»

Виконавець: **Суворов Дмитро**
Група: ЦІ-31
Викладач: С. В. Суліма

Київ — 2026

Зміст

1 Мета та виконані завдання **2**

2 Тест-план **2**

2.1 Об’єкти та область 2

2.2 Стратегія 2

2.3 Середовище та критерії завершення 3

3 Ручні тест-кейси **3**

4 Автоматизовані unit-тести **3**

5 Автоматизовані REST API-тести **4**

6 Запуск і покриття **4**

7 Баг-репорти та виправлення **5**

7.1 BUG-001 — PATCH стирав не передану примітку 5

7.2 BUG-002 — listener запускався при імпорті тестом 5

7.3 BUG-003 — неактуальна команда PostgreSQL 6

8 Підсумковий звіт **6**

9 Загальні висновки КП №1–3 **6**

10 Посилання **6**

1. Мета та виконані завдання

Мета роботи — спланувати й провести комплексне ручне та автоматизоване тестування наскрізного web-проєкту «Ритм», задокументувати дефекти, виконати виправлення і регресію.

- складено тест-план з областю, стратегією, ризиками та критеріями завершення;
- виконано 9 ручних позитивних і негативних тест-кейсів;
- реалізовано 12 unit-тестів бізнес-правил та валідації;
- реалізовано 12 інтеграційних REST API-тестів із реальною PostgreSQL;
- зафіксовано 3 дефекти, виконано виправлення та повну регресію;
- одна команда `npmtest` формує результат і звіт V8 coverage.

Виконавець	Суворов Дмитро
Група	ЦІ-31
Внесок	Планування, ручні кейси, unit/API-автоматизація, аналіз дефектів, виправлення, регресія і документація

2. Тест-план

2.1. Об’єкти та область

Перевірялися сім клієнтських сторінок, форми реєстрації/входу/запису, каталог, кабінет, адміністративний CRUD, маршрути `/api/auth`, `/api/trainings`, `/api/bookings`, `/api/categories`, `/api/health`, ролі member/admin, ізоляція власника та обмеження PostgreSQL.

Поза областю залишено production-навантаження, платіжні та зовнішні інтеграції, яких у застосунку немає.

2.2. Стратегія

1. Ручне функціональне тестування «чорної скриньки»: позитивні, негативні та межові сценарії у Chromium.
2. Unit-тестування «білої скриньки»: чисті функції та Zod-схеми модуля `server/validation.js` через `node:test`.
3. Інтеграційне API-тестування: Supertest викликає реальний Express, запити виконуються у PostgreSQL 17, тимчасові дані видаляються після набору.
4. Security-перевірки: 401 без JWT, 403 для member, відсутність mass assignment ролі, bcrypt-межі пароля.
5. Повна регресія після виправлень командою `npmtest` та повторенням ручних сценаріїв.

2.3. Середовище та критерії завершення

Node.js 26.13.0, npm 12.2.2, Express 5, PostgreSQL 17.11 у Docker Compose, Chromium, Linux. Вихідні критерії: щонайменше 5 ручних, 5 unit і 5 API-тестів; 100% Passed; відсутні відкриті Critical/Major; усі дефекти мають картки; сформовано coverage і test summary. Усі критерії виконано.

3. Ручні тест-кейси

Кожний кейс має ID, назву, передумови, кроки, очікуваний і фактичний результат та статус. Повні описи містяться у `docs/TEST-CASES.md`.

ID	Сценарій	Очікуваний результат	Фактичний результат	Статус
TC-01	Реєстрація з коректними даними	201, member, вхід	Створено, роль member	Passed
TC-02	Слабкий пароль	400, користувача немає	400, поле password	Passed
TC-03	Неправильний пароль	401, токена немає	401, спільне повідомлення	Passed
TC-04	Створення запису	201, дані у кабінеті/БД	Дані збігаються	Passed
TC-05	Неправильний телефон	400, запису немає	400, поле phone	Passed
TC-06	Часткове редагування	Інші поля збережені	Примітка збережена	Passed
TC-07	Admin API від member	403, без змін	403	Passed
TC-08	CRUD тренування admin	201/200/204/404	Коди й дані коректні	Passed
TC-09	Навігація desktop/mobile	Без 404/overflow	Усі сторінки доступні	Passed

4. Автоматизовані unit-тести

Unit-набір містить 12 тестів: межі PostgreSQL ID, normalizing email/імені, мінімальну довжину й bcrypt-межу 72 байти, strict-схему проти підміни ролі, валідні/минулі/неіснуючі дати, формат телефону, default примітки, тривалість і ціну тренування.

Лістинг 1: Приклад unit-тесту валідації ID

```
test("parseId rejects zero, fractions and injections", () => {
  for (const value of ["0", "1.5", "1 OR 1=1", "2147483648"])
```

```
   assert.throws(() => parseId(value), { status: 400 });
});
```

Модуль валідації винесено у `server/validation.js`; він не звертається до мережі/БД, тому тести швидкі та детерміновані.

5. Автоматизовані REST API-тести

Набір Supertest містить 12 сценаріїв: health, каталог, реєстрація, duplicate 409, неправильний пароль 401, захищений ресурс 401, validation 400, POST/PATCH/DELETE запису, 403 для member і JSON 404. Після прогону hook видаляє створені bookings/users.

Лістинг 2: Регресійний API-тест часткового оновлення

```
const response = await request(app)
  .patch(`/api/bookings/${bookingId}`)
  .set("Authorization", `Bearer ${token}`)
  .send({ phone: "+380671234567" })
  .expect(200);
assert.equal(response.body.data.note, "API regression");
```

Express app експортується без автоматичного відкриття порту; production-listener створюється лише при прямому запуску файлу.

6. Запуск і покриття

Лістинг 3: Єдина команда повної перевірки

```
docker compose up -d --wait db
npm run db:init
npm test
```

Результат: 24/24 автоматизованих тестів Passed. Загальне покриття: lines 80,44%, branches 78,46%, functions 67,57%. Для `server/validation.js`: 100% lines/branches/functions.

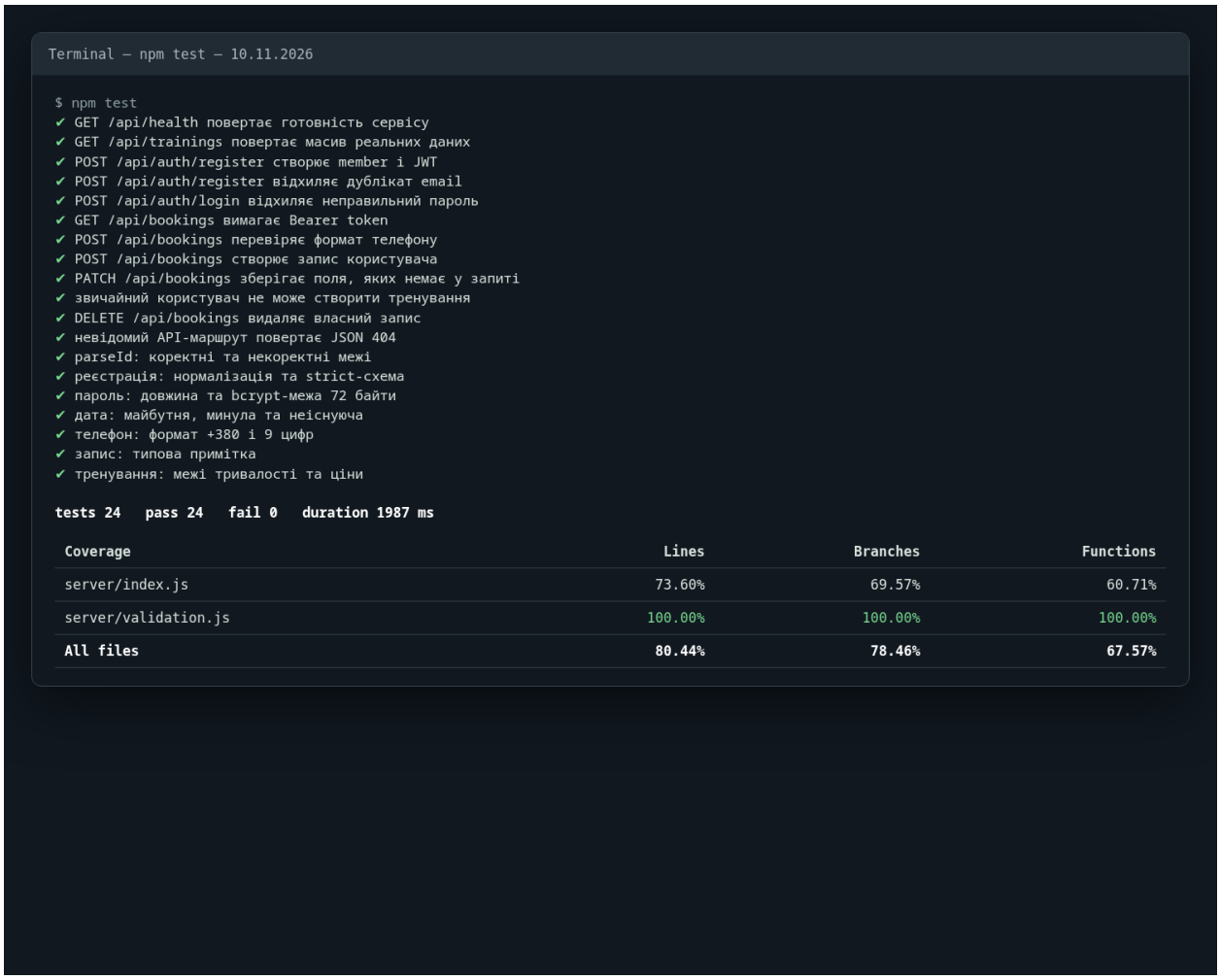


Рис. 1: Фактичний результат unit/API-тестів і coverage

7. Баг-репорти та виправлення

7.1. BUG-001 — PATCH стирав не передану примітку

Кроки: створити запис із приміткою; PATCH лише телефону; повторно отримати запис. **Очікувано:** змінюється лише телефон. **Фактично до виправлення:** default у partial-схемі підставляв порожню примітку. **Критичність:** Major. **Виправлення:** для PATCH note є optional без default. **Статус:** Fixed/Verified автоматизованим regression-тестом.

7.2. BUG-002 — listener запускався при імпорті тестом

Кроки: імпортувати Express app у Supertest при зайнятому порту. **Очікувано:** імпорт без мережевого listener. **Фактично:** можливий `EADDRINUSE`. **Критичність:** Major. **Виправлення:** експортовано `app`/`startServer()`, прямий запуск перевіряється окремо. **Статус:** Fixed/Verified 12 API-тестами.

7.3. BUG-003 — неактуальна команда PostgreSQL

Кроки: виконати psql з `gym_user/gym_db`. **Очікувано:** консоль БД. **Фактично:** role does not exist. **Критичність:** Minor. **Виправлення:** README використовує `ritm/ritm`, додано SQL та troubleshooting. **Статус:** Fixed/Verified.

Усі дефекти оформлено окремими задачами GitHub і додано до дошки. Повний шаблон баг-репортів: `docs/BUG-REPORTS.md`.

8. Підсумковий звіт

Набір	Всього	Passed	Failed	Успішність
Ручні	9	9	0	100%
Unit	12	12	0	100%
REST API	12	12	0	100%
Разом	33	33	0	100%

Дефекти: Critical — 0, Major — 2, Minor — 1, Trivial — 0; виправлено 3 із 3. Відкритих блокуючих дефектів немає. Навчальна версія відповідає критеріям завершення й готова до демонстрації. Production-публікація додатково потребуватиме HTTPS, зовнішнього сховища секретів, backup, monitoring і load testing.

9. Загальні висновки КП №1–3

У КП №1 створено семантичний адаптивний клієнт. У КП №2 додано нормалізовану PostgreSQL, REST API, bcrypt/JWT, рольовий доступ і Fetch-інтеграцію. У КП №3 якість зафіксовано відтворюваним тест-планом, ручними кейсами, 24 автоматизованими тестами, coverage та регресією. Наскрізний проєкт має повну історію розвитку і готовий до захисту.

10. Посилання

- Репозиторій: `https://github.com/Dima018/Gym_Website`
- Дошка задач: `https://github.com/users/Dima018/projects/13`
- Тест-план: `docs/TEST-PLAN.md`
- Тест-кейси: `docs/TEST-CASES.md`
- Баг-репорти: `docs/BUG-REPORTS.md`
- Test summary: `docs/TEST-SUMMARY.md`